Django Security Review

Educational Bug Bounty Series • Authorized Security Research Only

■ Django Security Review — Debug Exposure & Endpoint Discovery

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Review only authorized deployments. Do not enumerate large path lists or access debug pages containing secrets on production systems.*

■ Executive Summary

Django applications can expose sensitive implementation details when debug features are enabled or when operational endpoints are unintentionally public. The supplied one-page PDF references Django debug-mode exposure, a debug panel, a wordlist, and example API/health-check paths.

■ Core Testing Methodology

1. **Identify the framework.** Use passive evidence and authorized documentation to determine whether the application uses Django.
2. **Check debug configuration.** Verify in deployment configuration that debug mode is disabled in production.
3. **Review error handling.** Trigger only benign, expected errors in a staging environment and confirm responses do not expose tracebacks, environment variables, or secrets.
4. **Review operational endpoints.** Inventory documented health checks, API routes, and administrative endpoints.
5. **Check access control.** Ensure internal dashboards and sensitive API routes require authentication and appropriate roles.
6. **Use bounded discovery.** If route discovery is allowed, use a small, approved list and respect rate limits.
7. **Check deployment separation.** Ensure development tools and test endpoints are not included in production builds.
8. **Document and retest.** Record configuration evidence and sanitized responses.

■ High-Value Bug Bounty Tips & Edge Cases

- **Debug mode:** Production debug pages can expose source paths, settings, and request metadata.
- **Debug panels:** Developer tooling should not be reachable by untrusted users in production.
- **Health checks:** Health endpoints should return only the minimum operational information needed.
- **API route inventory:** The source lists API, reporting, transaction, organization, and document-like route patterns. Treat these as examples for reviewing your own route inventory, not a target-specific wordlist.
- **Source limitation:** The PDF is a short reference list and does not demonstrate a vulnerability in a particular Django application.

■■ Educational Checklists & Safe Examples

```
Application: [Your-Domain]
Framework: [Confirmed / Suspected Django]
Production DEBUG: [False / Unknown]
Debug details exposed: [Yes/No]
Sensitive route: [Redacted]
Authentication required: [Yes/No]
Evidence: [Redacted]
```

- [ ] `DEBUG` is disabled in production.
- [ ] `ALLOWED_HOSTS` is configured correctly.
- [ ] Secrets are stored outside source control.

- [ ] Error pages do not expose settings or tracebacks.
- [ ] Health checks reveal minimal information.
- [ ] Admin and API routes enforce authorization.
- [ ] Development tools are excluded from production.

■■ Mitigation & Remediation

- Set `DEBUG = False` in production and configure safe error pages.
- Configure `ALLOWED_HOSTS` narrowly and use HTTPS.
- Keep secrets in a managed environment or secret store, not source control.
- Restrict admin and operational endpoints with authentication, network controls, and least privilege.
- Minimize health-check output and protect diagnostic tools.
- Maintain dependencies and add deployment checks that fail if unsafe debug settings are enabled.

■ Source Notes

Based on the supplied one-page *Django.pdf*. It references remote-code-execution/debug-mode reading, a Django debug-panel report, a wordlist, and route examples including API, health-check, transaction, reporting, organization, and document paths. This rewrite treats those items as review prompts and does not reproduce external exploit instructions.

Educational use only. Test ethically and stay within scope.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.